Name : B.uma sankar
Project title : Student Academic Performance Prediction Project(minor)

Submitting to:
SprintM technologies pvt .ltd
(07-08-2025)

1. Introduction

This project is aimed at predicting the academic performance of students based on several factors such as attendance, past scores, study habits, and socio-economic indicators. We utilize machine learning (Random Forest) for prediction and visualize data using Plotly in a Gradio web dashboard.

2. Data Preprocessing

The dataset, `student_performance_data_for_tableau.csv`, contains features such as Attendance, Past Scores, Participation, Study Hours, Parent Education, Income Level, and Internet Access. Categorical values were label-encoded after filling missing values and normalizing inconsistent entries (like converting 'None' to 'No Formal Education').

3. Model Training

A Random Forest Classifier was trained using Scikit-learn. The dataset was split into 80% training and 20% testing. The target variable is 'Performance'. LabelEncoders were used for encoding categorical variables for compatibility with the model.

4. Prediction Interface

A Gradio-based interface allows users to input student details and get predicted performance levels instantly. Sliders and dropdowns ensure user-friendly input formatting.

5. Visualization Dashboard

A Plotly histogram embedded in a Gradio tab displays the distribution of student performance across different income levels. This helps in identifying socioeconomic patterns and trends visually.

6. Error Handling

The model gracefully handles unseen label errors (e.g., 'None') by mapping them to valid categories before encoding. This ensures the prediction module does not crash during runtime.

7. Technologies Used

- Python
- Pandas, NumPy, Scikit-learn
- Gradio for Web UI
- Plotly for Data Visualizations

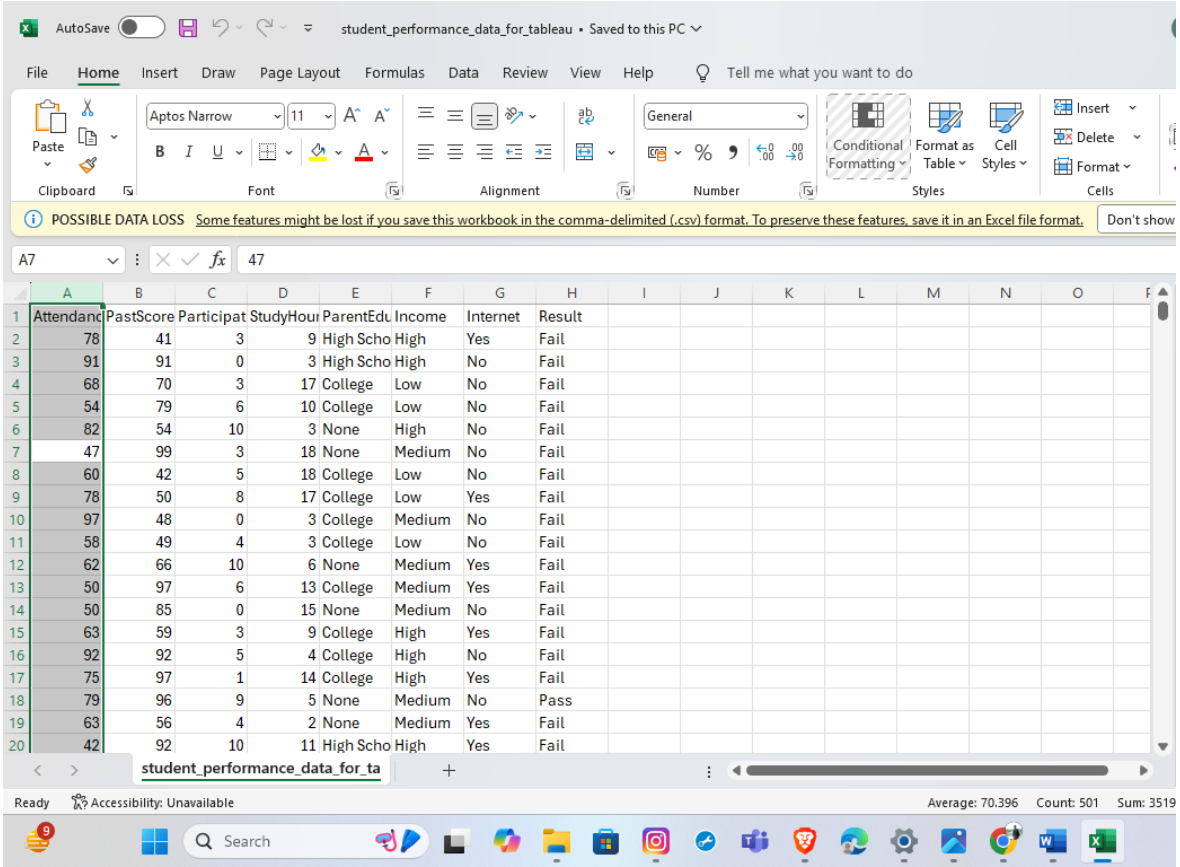
- Machine learning techniques like logistic regression
- Train and test
-

8. Future Improvements

- Include deep learning models for improved accuracy - Add gender, location, school type as features - Integrate Tableau dashboards for enterprise-level deployment - Support CSV upload for bulk predictions

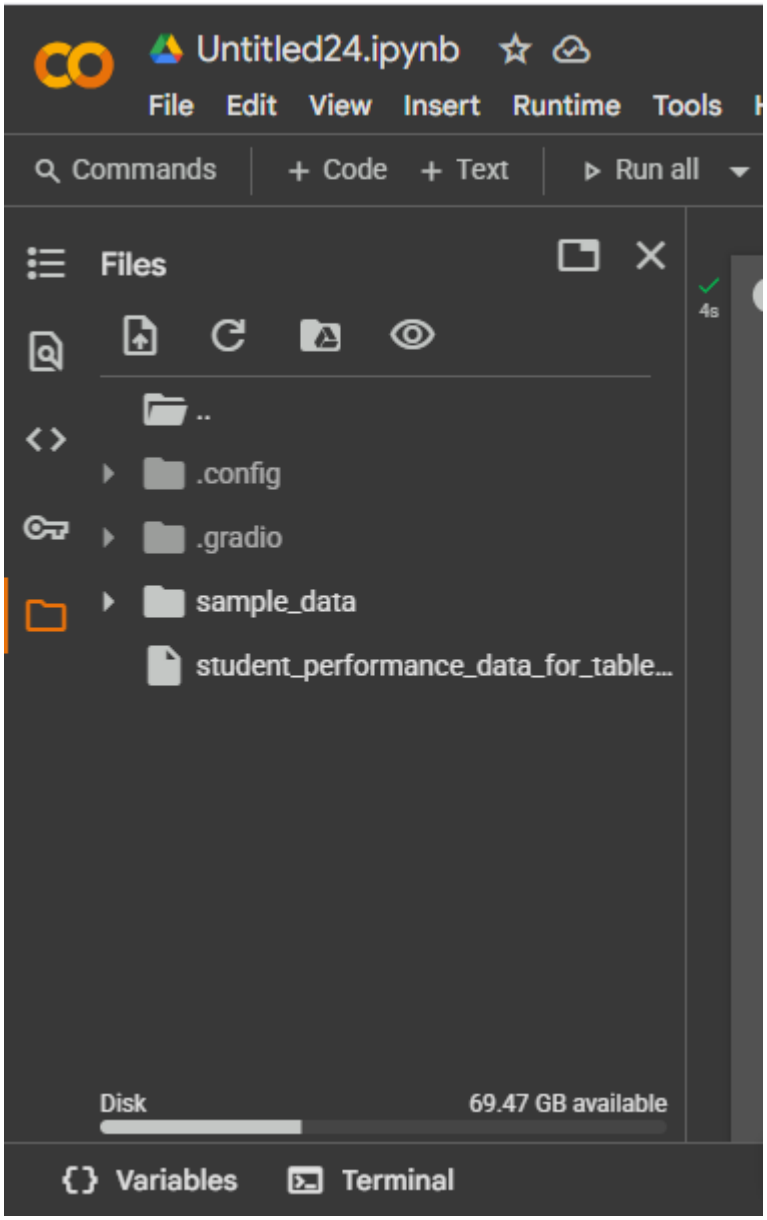
Step by step implementation of project code :

Step-1:collect the proper data base from the source like ms.excel or some where you want

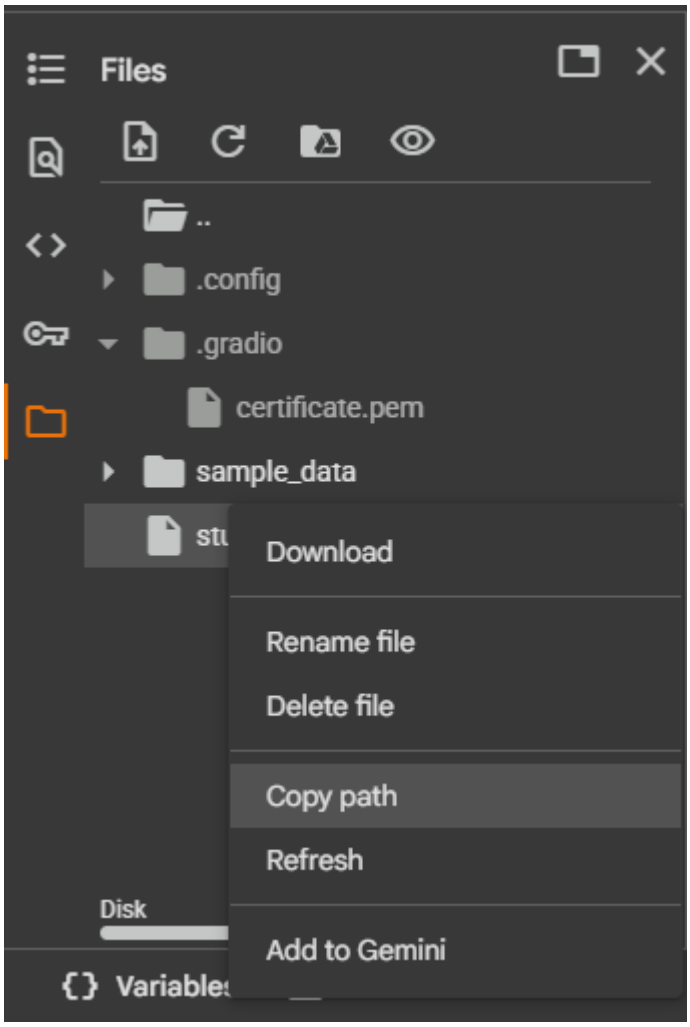


	A	B	C	D	E	F	G	H
1	Attendanc	PastScore	Participat	StudyHour	ParentEdu	Income	Internet	Result
2	78	41	3	9	High Scho	High	Yes	Fail
3	91	91	0	3	High Scho	High	No	Fail
4	68	70	3	17	College	Low	No	Fail
5	54	79	6	10	College	Low	No	Fail
6	82	54	10	3	None	High	No	Fail
7	47	99	3	18	None	Medium	No	Fail
8	60	42	5	18	College	Low	No	Fail
9	78	50	8	17	College	Low	Yes	Fail
10	97	48	0	3	College	Medium	No	Fail
11	58	49	4	3	College	Low	No	Fail
12	62	66	10	6	None	Medium	Yes	Fail
13	50	97	6	13	College	Medium	Yes	Fail
14	50	85	0	15	None	Medium	No	Fail
15	63	59	3	9	College	High	Yes	Fail
16	92	92	5	4	College	High	No	Fail
17	75	97	1	14	College	High	Yes	Fail
18	79	96	9	5	None	Medium	No	Pass
19	63	56	4	2	None	Medium	Yes	Fail
20	42	92	10	11	High Scho	High	Yes	Fail

Step-2 : Then upload(mount) downloaded csv file into google colab notebook



Step-3: copy the path of file which we uploaded collab notebook



Step-4: first of all we need to import all the libraries we need for our model

```
# first import libarires as per project
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
import gradio
import plotly.express as px
```

(module) express

Step-5: Then load the data csv file and read file while using “pd.read_csv” command

```
# Then load your provided CSV
data = pd.read_csv("/content/student_performance_data_for_tableau.csv")
```

Step-6:clean data for our modifications

```
# Clean columns if needed
data.columns = data.columns.str.strip()
```

Step-7:identify preview values and missing values

```
# Preview unique values
print("Unique ParentEdu:", data['ParentEdu'].unique())
print("Unique IncomeLevel:", data['Income'].unique())
print("Unique InternetAccess:", data['Internet'].unique())
```

```
# Map any missing or unknown categories
data['ParentEdu'] = data['ParentEdu'].replace({'None':'No Formal Education'})
data['ParentEdu'] = data['ParentEdu'].fillna('No Formal Education')
data['Income'] = data['Income'].fillna('Low')
data['Internet'] = data['Internet'].fillna('No')
```

Step-8: Apply encoding and decoding techniques to make predictionsusing x , y training and testing data

X=independent variables

Y=dependent variables

```
# Encode
le_parent = LabelEncoder()
le_income = LabelEncoder()
le_internet = LabelEncoder()
le_result = LabelEncoder() # Added LabelEncoder for the target variable

data['ParentEdu'] = le_parent.fit_transform(data['ParentEdu'])
data['Income'] = le_income.fit_transform(data['Income'])
data['Internet'] = le_internet.fit_transform(data['Internet'])
data['Result'] = le_result.fit_transform(data['Result']) # Encoded the target variable

X = data.drop("Result", axis=1) # Changed target column to 'Result'
y = data["Result"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier()
model.fit(X_train, y_train)
```

```
# Prediction interface
def predict(Attendance, PastScores, ParticipationScore, StudyHours, ParentEducation, IncomeLevel, InternetAccess):
    try:
        # Map 'None' safely
        if ParentEducation == "None":
            ParentEducation = "No Formal Education"
        # Encode
        pe = le_parent.transform([ParentEducation])[0]
        ie = le_income.transform([IncomeLevel])[0]
        ia = le_internet.transform([InternetAccess])[0]

        input_df = pd.DataFrame({
            'Attendance': [Attendance],
            'PastScores': [PastScores],
            'Participation': [ParticipationScore],
            'StudyHours': [StudyHours],
            'ParentEdu': [pe],
            'Income': [ie],
            'Internet': [ia]
        })

        pred = model.predict(input_df)[0]
```

```
        # Decode the prediction back to original class label
        predicted_performance = le_result.inverse_transform([pred])[0]
        return f"Predicted Performance: {predicted_performance}"

    except Exception as e:
        return f"Prediction failed: {str(e)}"
```

Step-9: After visualize the result into dashboard

Untitled24.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

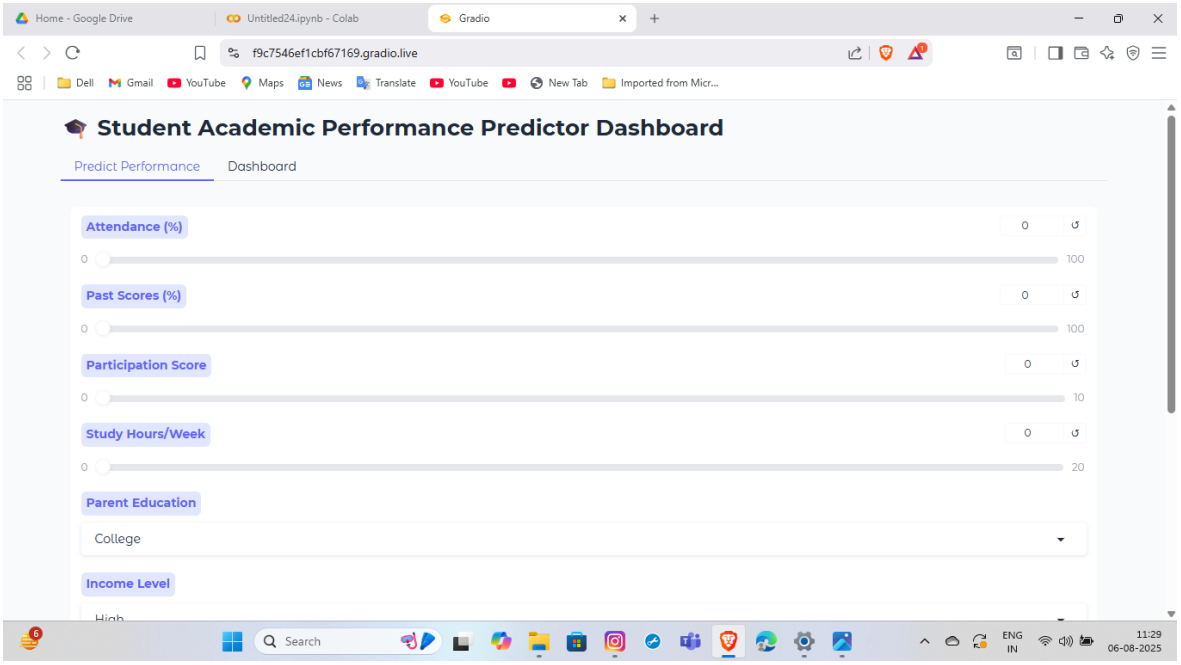
```
# Dashboard visualization
def dashboard():
    # Decode the 'Result' column for visualization
    data['Result_decoded'] = le_result.inverse_transform(data['Result'])
    data['Income_decoded'] = le_income.inverse_transform(data['Income'])
    fig = px.histogram(data, x="Result_decoded", color="Income_decoded", barmode="group", title="Student Performance Distribution")
    fig.update_layout(width=800, height=500)
    return fig


with gr.Blocks(theme=gr.themes.Soft()) as demo:
    gr.Markdown("# 🎓 Student Academic Performance Predictor Dashboard")

    with gr.Tab("Predict Performance"):
        Attendance = gr.Slider(0, 100, step=1, value=80, label="Attendance (%)")
        PastScores = gr.Slider(0, 100, step=1, value=70, label="Past Scores (%)")
        ParticipationScore = gr.Slider(0, 10, step=1, value=5, label="Participation Score")
        StudyHours = gr.Slider(0, 20, step=1, value=5, label="Study Hours/Week")

        ParentEducation = gr.Dropdown(
            choices=list(le_parent.classes_),
            label="Parent Education",
            value=le_parent.classes_[0]
        )
```

Step-10: Plot

```
    # plot.render(dashboard) # Remove this line

    gr.Markdown("**Professional ERP-like interface built with Gradio + Plotly.**")

demo.launch()
```

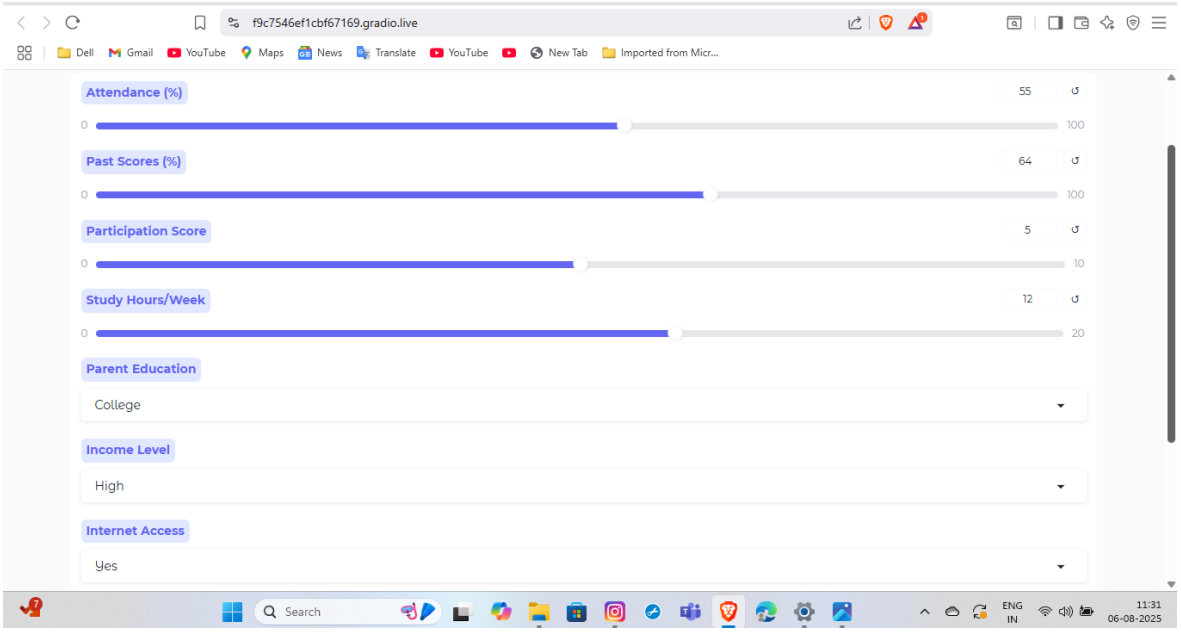
OUTPUT:

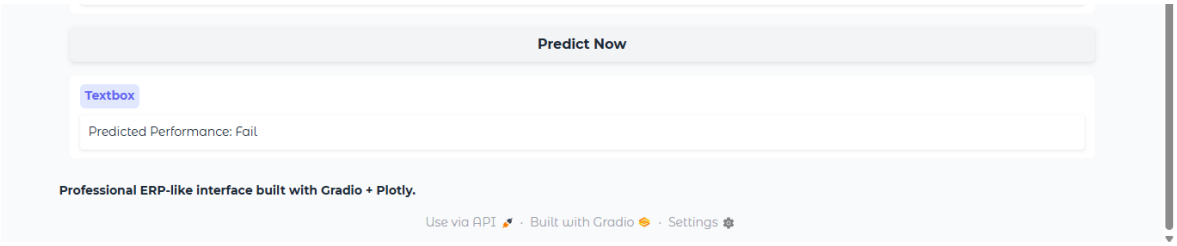
```
Unique ParentEdu: ['High School' 'College' nan]
Unique IncomeLevel: ['High' 'Low' 'Medium']
Unique InternetAccess: ['Yes' 'No']
It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True` (you can turn this off by sett

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://5d67cddcdd5ba6bd1b.gradio.live
```

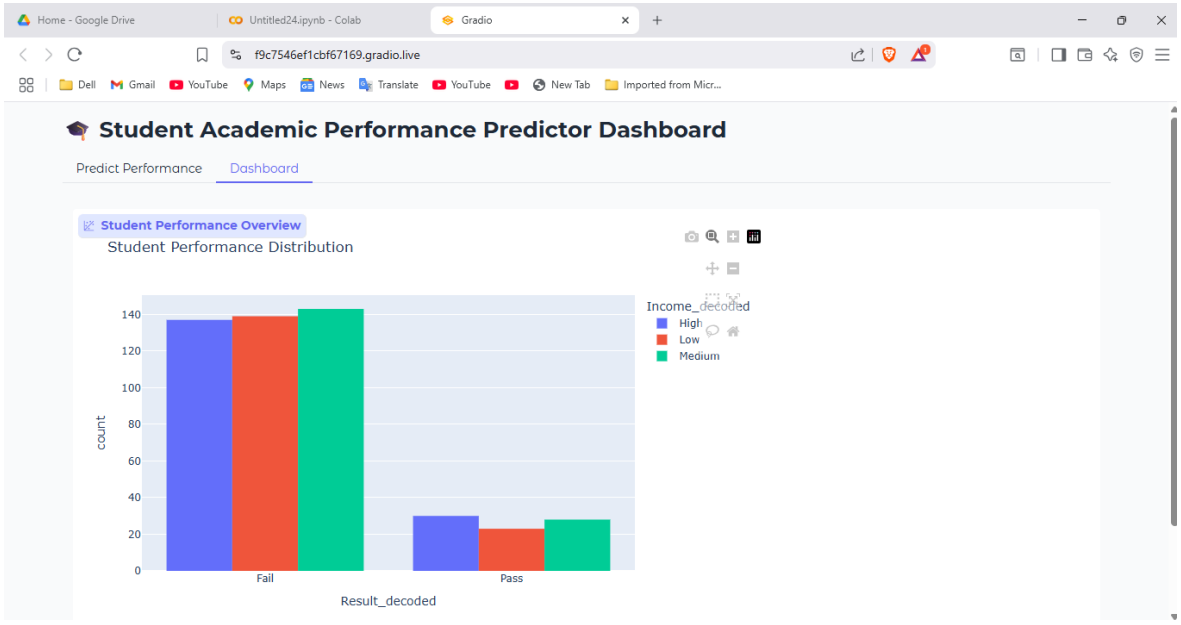


->compare model with various results:

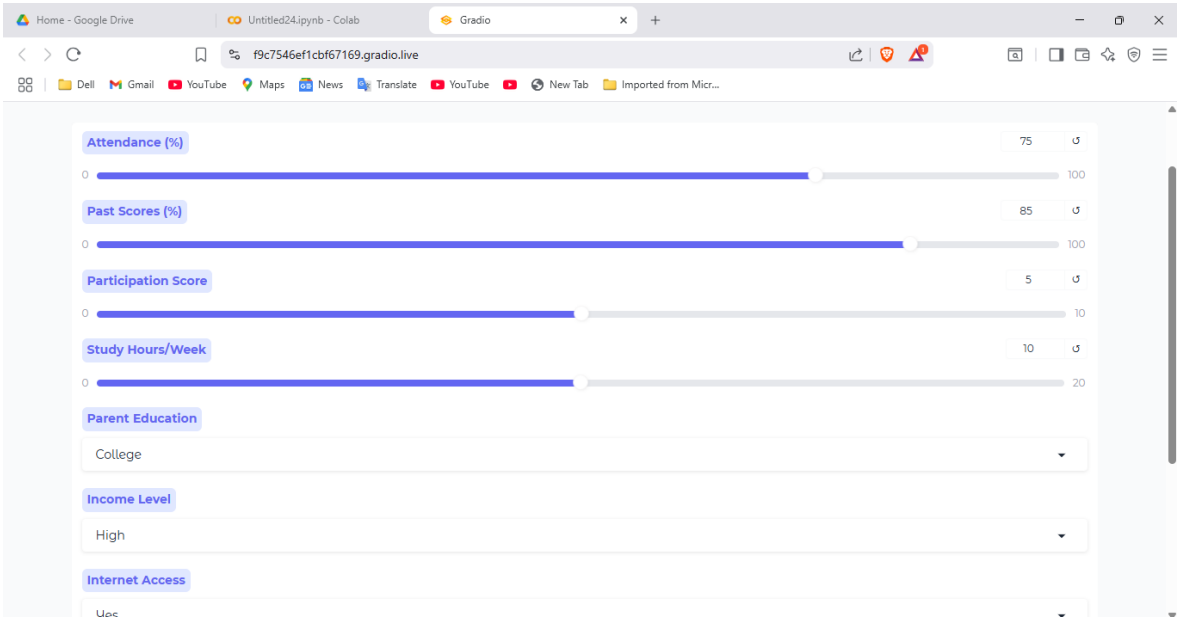
Fail: According to model below 75% feature are failed

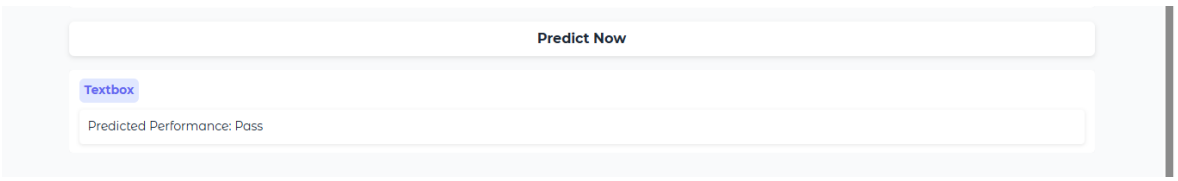


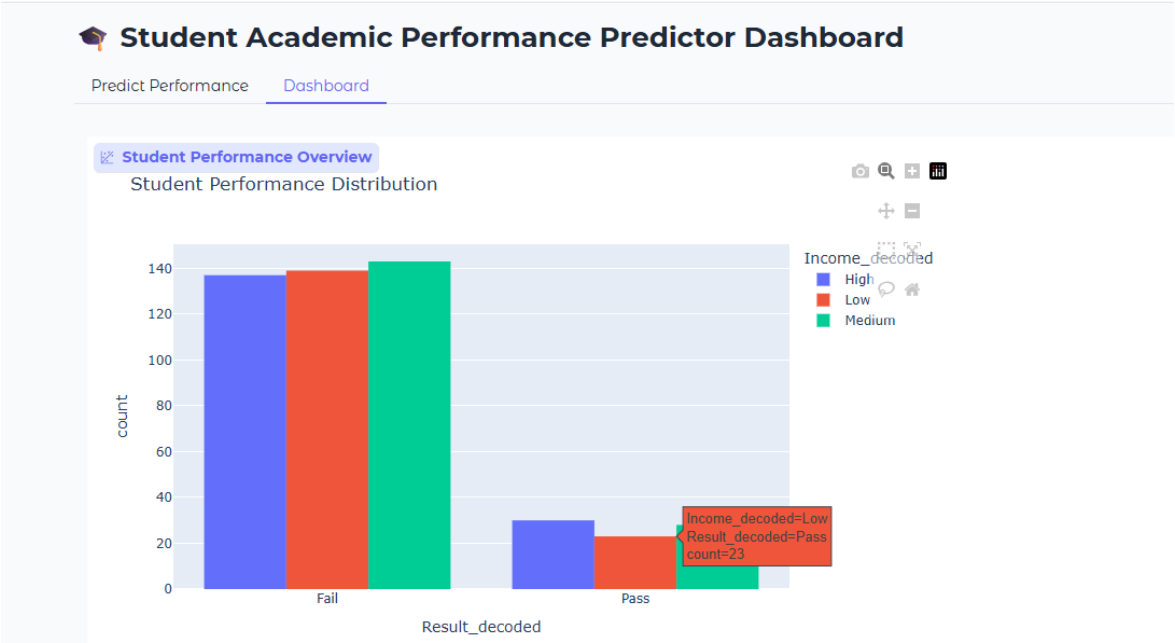




Pass : According to model 75% feature are pass






Student Academic Performance Predictor Dashboard
Predict Performance
Dashboard
Student Performance Overview
Student Performance Distribution
Income_decoded
High
Low
Medium
count
140
120
100
80
60
40
20
0
Fail
Pass
Result_decoded
Income_decoded=Low
Result_decoded=Pass
count=23

THE END